

1.5.1 Postfix

Postfix operators are operators that are suffixed to an expression.

operand++;

This causes the value of the *operand* to be returned. After the result is obtained, the value of the operand is incremented by 1.

operand--;

This is the same but the value of the operand is decremented by 1.

Examples:

```
int joe=3;
joe++;
```

The value of `joe` is now 4.

```
printf("%i",joe++);
```

This outputs the number 4. The value of `joe` is now 5.

1.5.2 Unary and Prefix

Prefix operators are operators that are prefixed to an expression.

++operand;

This causes the value of the operand to be incremented by 1. Its new value is then returned.

--operand;

This is the same but the value of the operand is decremented by 1.

!operand

Returns the logical NOT operation on the operand. A true operand returns false, a false operand returns true. Also known as the bang operand.

~operand

Returns the compliment of the operand. The returned value is the operand with its bits reversed (1's become 0's, 0's become 1's).

Examples:

```
int bart=7;
printf("%i",--bart);
```

This outputs the number 6. The value of `bart` is now 6.

```
int lisa=1;
printf("%i",!lisa);
```

This outputs 0 (false).

1.5.3 Normal

There are several normal operators which return the result defined for each:

expression1 + *expression*

The result of this is the sum of the two expressions.

expression1 - *expression2*

The result of this is the value of *expression2* subtracted from *expression1*.

expression1 * *expression2*

The result of this is the value of *expression1* multiplied by *expression2*.

expression1 / *expression2*

The result of this is the value of *expression1* divided by *expression2*.

expression1 % *expression2*

The result of this is the value of the remainder after dividing *expression1* by *expression2*. Also called the modulo operator.

expression1 & *expression2*

Returns a bitwise AND operation done on *expression1* and *expression2*. The result is a value the same size as the expressions with its bits modified using the following rules: Both bits must be 1 (on) to result in 1 (on), otherwise the result is 0 (off).

e1	e2	Result
0	0	0
0	1	0
1	0	0
1	1	1

expression1 | *expression2*

Returns a bitwise OR operation done on *expression1* and *expression2*. The result is a value the same size as the expressions with its bits modified using the following rules: Both bits must be 0 (off) to result in 0 (off), otherwise the result is 1 (on).

e1	e2	Result
0	0	0
0	1	1
1	0	1
1	1	1

expression1 ^ *expression2*

Returns a bitwise XOR operation done on *expression1* and *expression2*. The result is a value the same size as the expressions with its bits modified using the following rules: If both bits are the same, then the result is 0 (off), otherwise the result is 1 (on).

e1	e2	Result
0	0	0
0	1	1
1	0	1
1	1	0

expression1 >> *shift_value*

Returns *expression1* with its bits shifted to the right by the *shift_value*. The leftmost bits are replaced with zeros if the value is nonnegative or unsigned. This result is the integer part of *expression1* divided by 2 raised to the power of *shift_value*. If *expression1* is signed, then the result is implementation specific.

expression1 << *shift_value*

Returns *expression1* with its bits shifted to the left by the *shift_value*. The rightmost bits are replaced with zeros. This result is the value of *expression1* multiplied by the value of 2 raised to the power of *shift_value*. If *expression1* is signed, then the result is implementation specific.

1.5.4 Boolean

The boolean operators return either 1 (true) or 0 (false).

expression1 && *expression2*

Returns the logical AND operation of *expression1* and *expression2*. The result is 1 (true) if both expressions are true, otherwise the result is 0 (false).

e1	e2	Result
0	0	0
0	1	0
1	0	0
1	1	1

expression1 || *expression2*

Returns the logical OR operation of *expression1* and *expression2*. The result is 0 (false) if both expressions are false, otherwise the result is 1 (true).

e1	e2	Result
0	0	0
0	1	1
1	0	1
1	1	1

expression1 < *expression2*

Returns 1 (true) if *expression1* is less than *expression2*, otherwise the result is 0 (false).

expression1 > *expression2*

Returns 1 (true) if *expression1* is greater than *expression2*, otherwise the result is 0 (false).

expression1 <= *expression2*

Returns 1 (true) if *expression1* is less than or equal to *expression2*, otherwise the result is 0 (false).

expression1 >= *expression2*

Returns 1 (true) if *expression1* is greater than or equal to *expression2*, otherwise the result is 0 (false).

expression1 == *expression2*

Returns 1 (true) if *expression1* is equal to *expression2*, otherwise the result is 0 (false).

expression1 != *expression2*

Returns 1 (true) if *expression1* is not equal to *expression2*, otherwise the result is 0 (false).

1.5.5 Assignment

An assignment operator stores the value of the right expression into the left expression.

expression1 = *expression2*

The value of *expression2* is stored in *expression1*.

expression1 *= *expression2*

The value of *expression1* times *expression2* is stored in *expression1*.

expression1 /= *expression2*

The value of *expression1* divided by *expression2* is stored in *expression1*.

expression1 %= *expression2*

The value of the remainder of *expression1* divided by *expression2* is stored in *expression1*.

expression1 += *expression2*

The value of *expression1* plus *expression2* is stored in *expression1*.

expression1 -= *expression2*

The value of *expression1* minus *expression2* is stored in *expression1*.

expression1 <<= *shift_value*

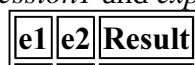
The value of *expression1*'s bits are shifted to the left by *shift_value* and stored in *expression1*.

expression1 >>= *shift_value*

The value of *expression1*'s bits are shifted to the right by *shift_value* and stored in *expression1*.

expression1 &= *expression2*

The value of the bitwise AND of *expression1* and *expression2* is stored in *expression1*.



0	0	0
0	1	0
1	0	0
1	1	1

expression1 ^= *expression2*

The value of the bitwise XOR of *expression1* and *expression2* is stored in *expression1*.

e1	e2	Result
0	0	0
0	1	1
1	0	1
1	1	0

expression1 |= *expression2*

The value of the bitwise OR of *expression1* and *expression2* is stored in *expression1*.

e1	e2	Result
0	0	0
0	1	1
1	0	1
1	1	1

1.5.6 Precedence

The operators have a set order of precedence during evaluation. Items encapsulated in parenthesis are evaluated first and have the highest precedence. The following chart shows the order of precedence with the items at the top having highest precedence.

Operator	Name
!	Logical NOT. Bang.
++ --	Increment and decrement operators.
* / %	Multiplicative operators.
+ -	Additive operators.
<< >>	Shift operators.
< > <= >=	Inequality comparators.
== !=	Equality comparators
&	Bitwise AND.
^	Bitwise XOR.
	Bitwise OR.
&&	Logical AND.
	Logical OR.

?: Conditional.
= *op*= Assignment.

Examples:

`17 * 5 + !(1+1) && 0`

Evaluates to 0 (false).

`5+7<4`

Evaluates to 1 (true).

`a<b<c`

Same as `(a<b)<c`.